

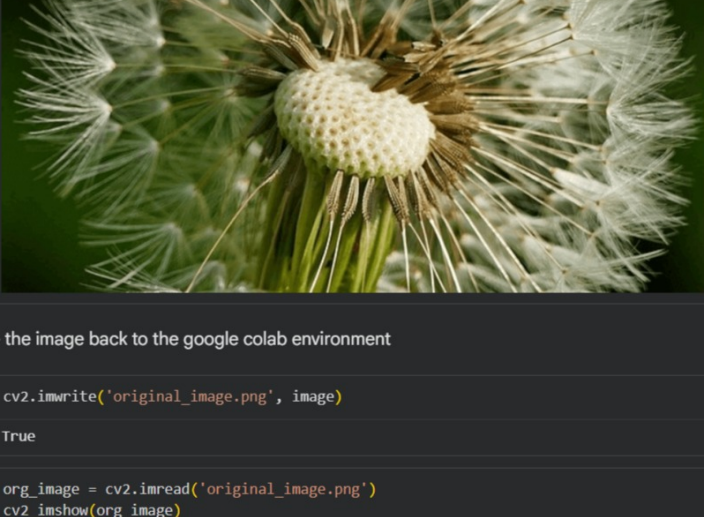
import library

```
import cv2
from google.colab.patches import cv2_imshow
import numpy as np
```

load and display image

```
#load image
image = cv2.imread('image_02.png')

#display image
cv2_imshow(image)
```



write the image back to the google colab environment

```
cv2.imwrite('original_image.png', image)

True
```

```
org_image = cv2.imread('original_image.png')
cv2_imshow(org_image)
```



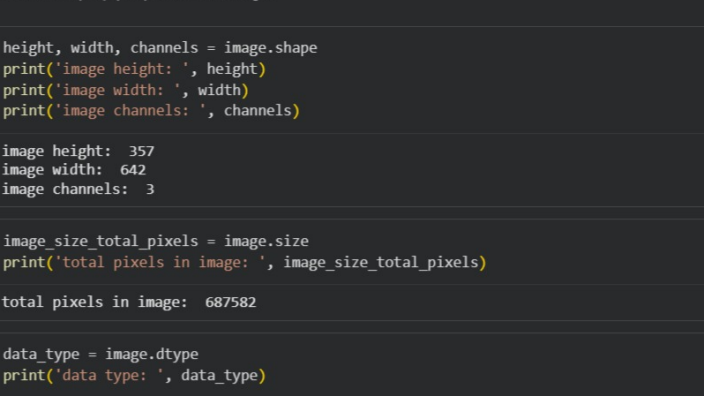
modify original image

```
for x in range(100):
    for y in range(100):
        image[x,y] = [0,154,255]

cv2.imwrite('modified_image.png', image)

True
```

```
modified_image = cv2.imread('modified_image.png')
cv2_imshow(modified_image)
```



access and display properties in image

```
height, width, channels = image.shape
print('image height: ', height)
print('image width: ', width)
print('image channels: ', channels)

image height: 357
image width: 642
image channels: 3

image_size_total_pixels = image.size
print('total pixels in image: ', image_size_total_pixels)

total pixels in image: 687582

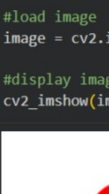
data_type = image.dtype
print('data type: ', data_type)

data type: uint8
```

ROI of 100*100 pixels

```
roi = image [0:100, 0:100] #first 0:100 y(h) axis, second 0:100 x(w) axis
cv2.imwrite('ROI_image.png', roi)

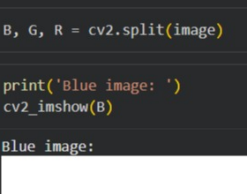
ROI_image = cv2.imread('ROI_image.png')
cv2_imshow(ROI_image)
```



load and display image

```
#load image
image = cv2.imread('opencv_logo.png')

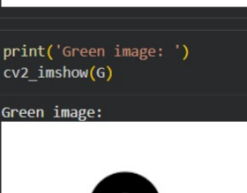
#display image
cv2_imshow(image)
```



```
B, G, R = cv2.split(image)

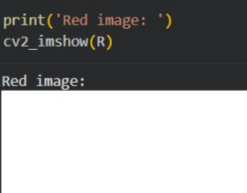
print('Blue image: ')
cv2_imshow(B)

Blue image:



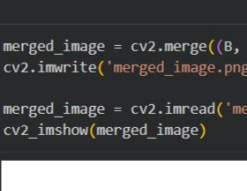
print('Green image: ')
cv2_imshow(G)

Green image:



print('Red image: ')
cv2_imshow(R)

Red image:


```

merge the sepearate colors channels in single image

```
merged_image = cv2.merge((B, G, R))
cv2.imwrite('merged_image.png', merged_image)

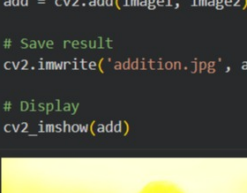
merged_image = cv2.imread('merged_image.png')
cv2_imshow(merged_image)
```



swap color channels

```
swaped_image = cv2.merge((G, R, B))
cv2.imwrite('swaped_image.png', swaped_image)

swaped_image = cv2.imread('swaped_image.png')
cv2_imshow(swaped_image)
```



Arithmetic Operations (Addition & Subtraction)

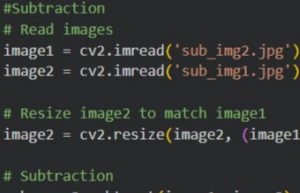
```
# Read images
image1 = cv2.imread('image2.jpg')
image2 = cv2.imread('image1.webp')

image2 = cv2.resize(image2, (image1.shape[1], image1.shape[0]))

# Perform addition
add = cv2.add(image1, image2)

# Save result
cv2.imwrite('addition.jpg', add)

# Display
cv2_imshow(add)
```



```
#Subtraction
# Read images
image1 = cv2.imread('sub_img2.jpg')
image2 = cv2.imread('sub_img1.jpg')

# Resize image2 to match image1
image2 = cv2.resize(image2, (image1.shape[1], image1.shape[0]))

# Subtraction
sub = cv2.subtract(image1, image2)

# Save output
cv2.imwrite('subtraction.jpg', sub)

# Display
cv2_imshow(sub)
```

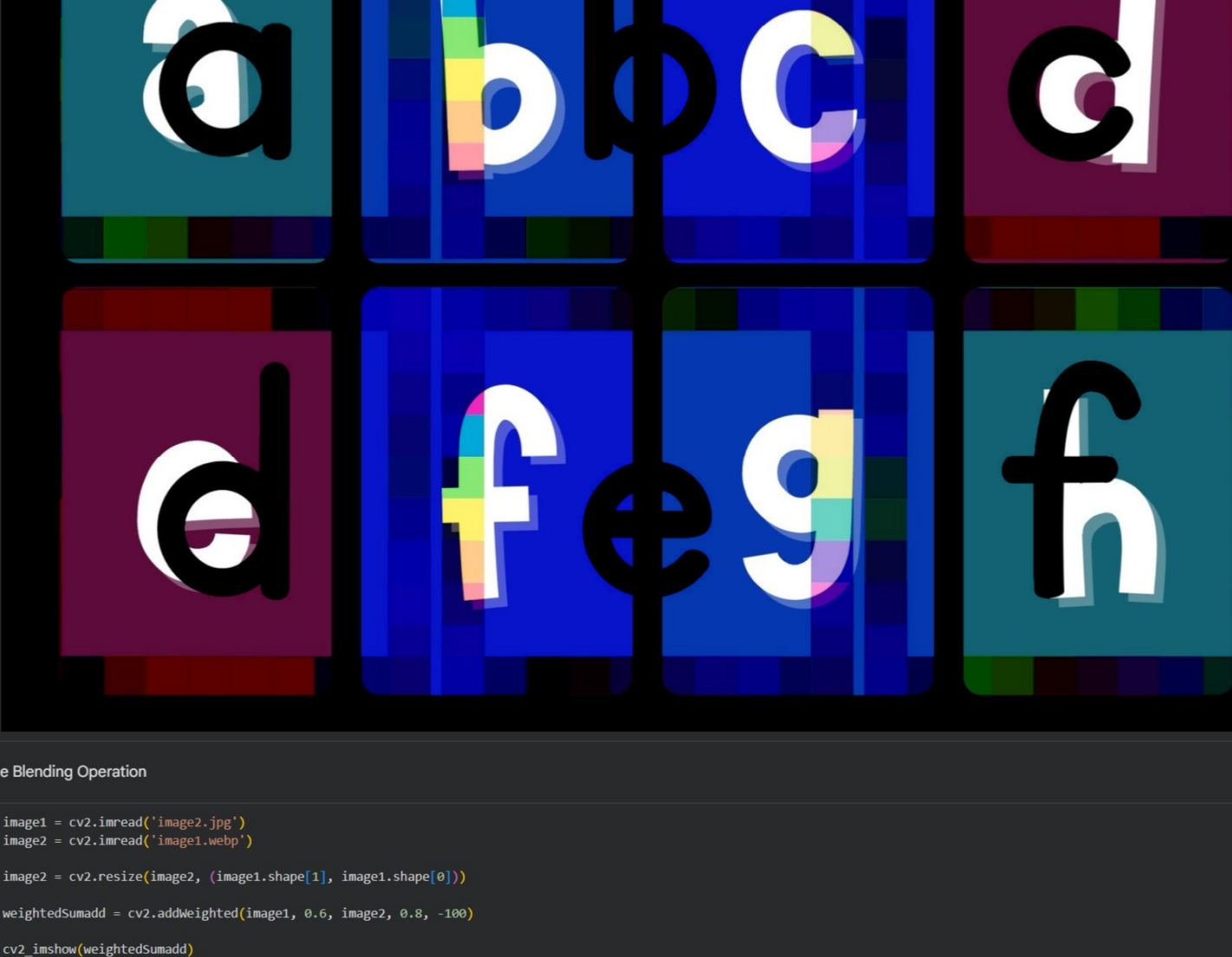


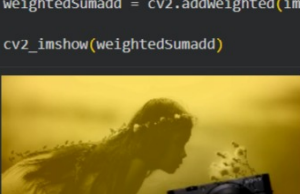
Image Blending Operation

```
image1 = cv2.imread('image2.jpg')
image2 = cv2.imread('image1.webp')

image2 = cv2.resize(image2, (image1.shape[1], image1.shape[0]))

weightedSumadd = cv2.addWeighted(image1, 0.6, image2, 0.8, -100)

cv2_imshow(weightedSumadd)
```

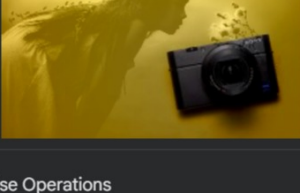


```
image1 = cv2.imread('image2.jpg')
image2 = cv2.imread('image1.webp')

image2 = cv2.resize(image2, (image1.shape[1], image1.shape[0]))

weightedSumadd = cv2.addWeighted(image1, 0.6, image2, 0.8, -75)

cv2_imshow(weightedSumadd)
```



```
image1 = cv2.imread('image2.jpg')
image2 = cv2.imread('image1.webp')

image2 = cv2.resize(image2, (image1.shape[1], image1.shape[0]))

weightedSumadd = cv2.addWeighted(image1, 0.6, image2, 0.8, -50)

cv2_imshow(weightedSumadd)
```

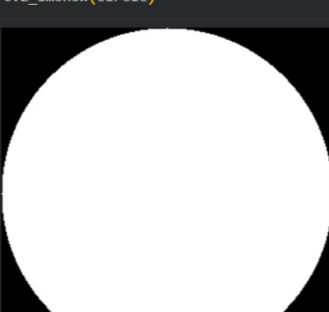


Bitwise Operations

```
#14a & 14b. Rectangle Image (300x300)
import numpy as np

# Create black image
rect = np.zeros((300, 300), dtype=np.uint8)

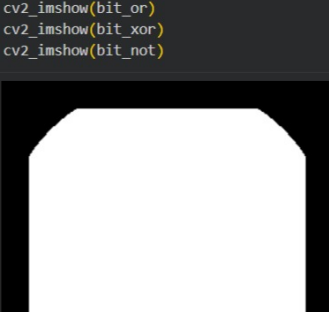
# Draw white filled rectangle
cv2.rectangle(rect, (25,25), (275,275), 255, -1) # (25,25) = top-left corner, (275,275) = bottom-right corner, 255 color, -1 = filled rectangle
cv2_imshow(rect)
```



```
#14c & 14d. Circle Image (300x300)
# Create black image
circle = np.zeros((300, 300), dtype=np.uint8)

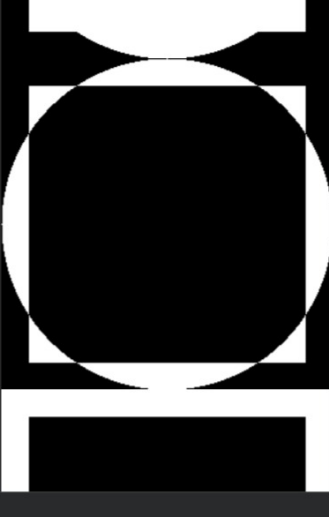
# Draw white filled circle
cv2.circle(circle, (150,150), 150, 255, -1)

cv2_imshow(circle)
```



```
#14e. Bitwise operations (AND, OR, XOR, NOT)
bit_and = cv2.bitwise_and(rect, circle)
bit_or = cv2.bitwise_or(rect, circle)
bit_xor = cv2.bitwise_xor(rect, circle)
bit_not = cv2.bitwise_not(rect)

# Display results
cv2_imshow(bit_and)
cv2_imshow(bit_or)
cv2_imshow(bit_xor)
cv2_imshow(bit_not)
```



Colab paid products - Cancel contracts here